Qu'est-ce qu'une pipeline CI/CD ?

Une développeuse pousse son code sur **Git**. Quelques minutes plus tard, elle reçoit une notification : les tests passent, l'image **Docker** est construite, le déploiement en staging est terminé. **Elle n'a rien fait manuellement**. C'est la pipeline CI/CD qui a tout orchestré, automatiquement.

Mais qu'est-ce qui distingue une pipeline d'un simple script shell ? Pourquoi cette abstraction est-elle devenue incontournable ?

> ***En bref** : Une pipeline CI/CD est un **système d'exécution gouverné** qui transforme du code source en logiciel déployable, de façon reproductible, traçable et observable. Ce n'est pas un script — c'est une infrastructure.*

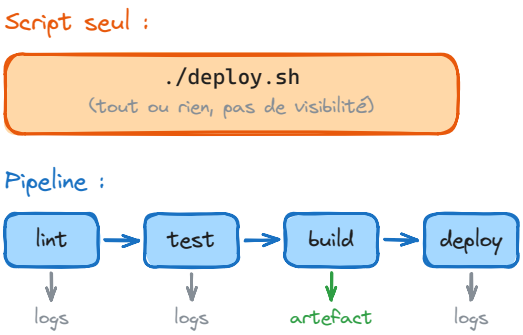
Pipeline n'est pas un script

Vous avez peut-être un script **Bash** qui compile et déploie votre application. **Ce n'est pas une pipeline**.

La différence ne tient pas à ce que fait le code, mais à **comment il s'exécute** et **ce qu'on peut en observer**.

Script	Pipeline
Tourne sur votre machine ou un serveur fixe	S'exécute dans un environnement contrôlé et isolé
Lit les variables d'environnement locales	Déclare explicitement ses besoins (secrets, artefacts)
Renvoie « ça marche » ou « ça plante »	Expose chaque étape, son statut, sa durée
Aucune trace une fois terminé	Conserve logs, artefacts, historique complet
Lancé manuellement ou par **cron**	Déclenché par des événements Git (push, PR, tag)

Un script peut être une étape d'une pipeline — mais la pipeline apporte l'orchestration, la gouvernance et la visibilité que le script seul ne peut pas fournir.



La structure d'exécution

Pour comprendre une pipeline, il faut d'abord installer quelques concepts. Prenons-les un par un, du plus petit au plus grand.

Jobs : l'unité atomique

Un **job** est la plus petite unité d'exécution. Imaginez-le comme un conteneur jetable : il démarre, fait son travail, puis disparaît. Chaque job s'exécute dans un environnement isolé et produit un résultat binaire : succès ou échec.

Le cycle de vie d'un job :

1. Préparation de l'environnement (image Docker, dépendances système)
2. Checkout du code source
3. Exécution des commandes (steps)
4. Upload des artefacts (si succès)
5. Nettoyage

Ce qui définit un job :

- **Isolation** — Chaque job démarre dans un environnement vierge. Ce qui s'est passé dans un job précédent ne pollue pas celui-ci.
- **Atomicité** — Un job réussit (exit code 0) ou échoue (exit code ≠ 0). Pas d'état intermédiaire.
- **Parallélisation** — Plusieurs jobs peuvent tourner en même temps s'ils ne dépendent pas les uns des autres.

Exemples concrets de jobs :

- **`lint`** — Vérifie le style du code
- **`unit-tests`** — Exécute les tests unitaires
- **`build-docker`** — Construit une image Docker
- **`deploy-staging`** — Déploie sur l'environnement de test

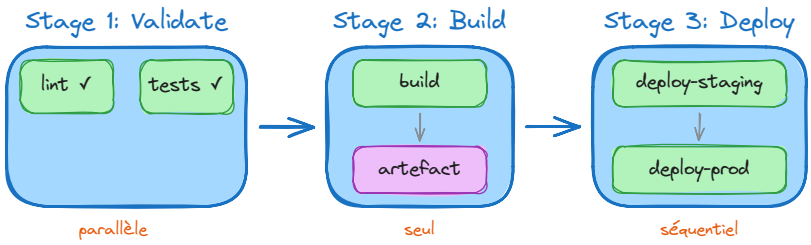
Chacun de ces jobs est **indépendant**. Si les tests plantent, ça n'empêche pas le linter de tourner (ils peuvent tourner en parallèle). Mais si les tests plantent, on ne construira pas l'image Docker (relation de dépendance).

Stages : organiser les jobs par phase

Un **stage** (ou phase) regroupe des jobs qui ont le même objectif. Les stages s'exécutent **dans l'ordre**, mais les jobs d'un même stage peuvent tourner **en parallèle**.

Exemple classique :

1. **Stage "Validate"** — **`lint`** et **`tests`** en parallèle
2. **Stage "Build"** — Construction de l'artefact (une fois les tests OK)
3. **Stage "Deploy"** — Déploiement progressif (staging puis production)



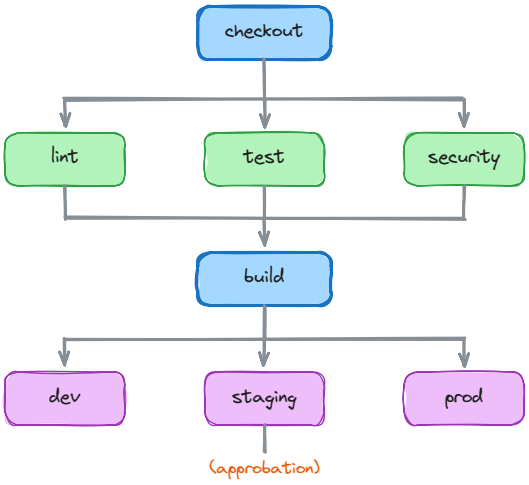
Pourquoi découper en stages ?

- **Fail fast** — Si **`lint`** échoue, inutile de lancer **`build`**
- **Lisibilité** — Visualisation claire du flux dans l'interface

- **Optimisation** — Parallélisation maximale au sein d'un stage

Le graphe d'exécution (DAG)

Une pipeline complexe forme un **graphe acyclique dirigé** (DAG). Les dépendances entre jobs définissent l'ordre d'exécution — pas seulement les stages.



Comprendre ce graphe permet d'**optimiser les temps d'exécution** : si deux jobs n'ont pas de dépendance entre eux, ils peuvent tourner en parallèle.

Runners et agents

Qui exécute le code ?

Un **runner** (**GitHub Actions**, **GitLab CI**) ou **agent** (**Jenkins**, **Azure DevOps**) est la **machine** qui exécute vos jobs. Le runner récupère le job depuis la plateforme, exécute les commandes, et renvoie le résultat.

Le flux d'exécution :

1. La plateforme CI/CD assigne un job au runner disponible
2. Le runner récupère le job (code, configuration)
3. Le job s'exécute (commandes, scripts)
4. Le runner renvoie les logs et le statut à la plateforme

Types de runners

Type	Avantages	Inconvénients
Hébergé (GitHub, **GitLab** SaaS)	Prêt à l'emploi, maintenance incluse	Ressources limitées, pas d'accès réseau interne
Auto-hébergé	Contrôle total, accès ressources internes	Maintenance et sécurité à gérer
Éphémère	Isolation maximale, pas de pollution entre jobs	Temps de démarrage plus long

Ce qui change selon le type de runner :

- **Sécurité** — Un runner partagé (public) peut exposer vos secrets si mal configuré. Un runner privé limite les risques.
- **Performance** — Accès à des ressources spécifiques (bases de données internes, GPU pour le ML).
- **Coût** — Minutes gratuites épuisées rapidement sur les runners publics si vous construisez souvent.

Isolation et sécurité des runners

Un job a accès à **tout ce que le runner peut voir** :

- Système de fichiers du runner
- Variables d'environnement
- Secrets injectés
- Réseau accessible depuis le runner

Risque : Si plusieurs projets partagent le même runner, un job malveillant peut potentiellement accéder aux données d'un autre projet. C'est pourquoi les runners **éphémères** (détruits après chaque job) sont recommandés pour les contextes sensibles.

Cache vs artefacts

La confusion entre cache et artefact est fréquente. Les deux stockent des données, mais leur usage est **fondamentalement différent**.

Cache : accélérer les builds

Le **cache** conserve des fichiers **entre exécutions** pour éviter de les recalculer. Typiquement : dépendances npm, modules Python, cache Maven.

Exemple concret :

- **Run 1** : `npm install` télécharge 300 packages → 3 minutes → sauvegarde le cache
- **Run 2** : `npm install` trouve le cache → 10 secondes

Caractéristiques du cache :

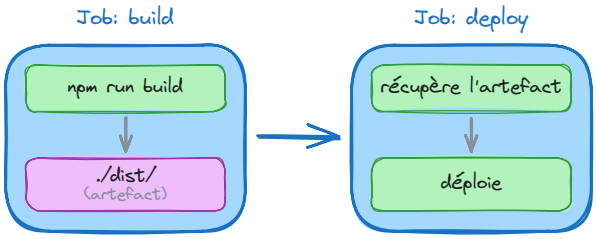
- Clé basée sur un hash (lockfile, branche)
- **Optionnel** — le job doit fonctionner même sans cache
- Partagé entre jobs et pipelines
- Peut devenir obsolète (et c'est normal)

Artefacts : transmettre des résultats

Un **artefact** est le **produit d'un job**, passé aux jobs suivants ou téléchargé par un humain. C'est le livrable de l'étape.

Pourquoi c'est important ?

Parce que les jobs sont **isolés**. Si le job **`build`** produit un fichier, le job **`deploy`** ne le verra pas — sauf si ce fichier est déclaré comme artefact.



Caractéristiques des artefacts :

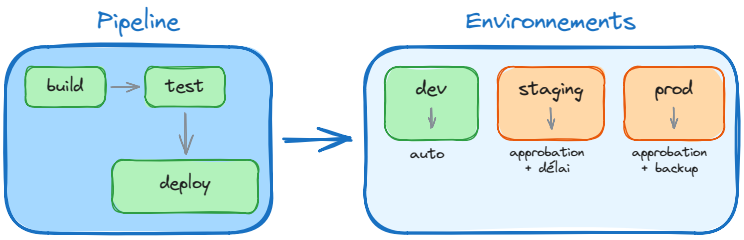
- Attachés à un job précis
- Durée de rétention configurable (30-90 jours par défaut)
- Téléchargeables après la pipeline
- **Obligatoires** pour les jobs dépendants

Tableau comparatif

Aspect	Cache	Artefact
But	Accélérer	Transmettre
Obligatoire	Non	Oui (si dépendance)
Persistance	Entre runs	Durée limitée
Exemple	**`node_modules/`**	**`dist/`**, **`report.xml`**
Si absent	Job plus lent	Job échoue

Environnements de déploiement

Un **environnement** représente une cible de déploiement : **`dev`**, **`staging`**, **`production`**. Les pipelines modernes permettent de définir des règles par environnement.



Règles de protection

Les environnements de production méritent des protections :

- **Approbation** — Un humain doit valider avant déploiement
- **Délai** — Attendre X minutes entre staging et prod (pour annuler si erreur)

- **Plages horaires** — Pas de déploiement le vendredi après-midi
- **Branches autorisées** — Seul `main` peut déployer en prod

Secrets par environnement

Chaque environnement a ses propres credentials :

- Base de données staging ≠ base de données prod
- Clés API de test ≠ clés API de production
- Tokens avec permissions réduites en dev

La séparation des secrets réduit l'impact d'une fuite : un secret staging compromis ne donne pas accès à la production.

Logs et observabilité

Ce que les logs doivent contenir

Un job bien loggé permet de comprendre ce qui s'est passé **sans relancer la pipeline** :

- **Commandes exécutées** — Exactement ce qui a été lancé
- **Sortie standard/erreur** — Output complet des commandes
- **Timestamps** — Quand chaque étape a commencé/fini
- **Variables d'environnement** — Contexte d'exécution (sauf secrets)
- **Versions** — Outils, dépendances, images utilisées

Masquage des secrets

Les plateformes CI/CD masquent automatiquement les secrets dans les logs. Mais attention aux fuites indirectes :

```
# ❌ Le secret peut fuiter via echo
echo "Token: $MY_TOKEN"

# ❌ Le secret peut fuiter via erreur curl
curl -H "Authorization: $TOKEN" https://api.example.com
# Si l'URL est invalide, l'erreur peut afficher le token

# ✓ Utiliser des fichiers temporaires
echo "$TOKEN" > /tmp/token.txt
curl -H "Authorization: $(cat /tmp/token.txt)" https://api.example.com
```

Métriques essentielles

Une pipeline en bonne santé expose ses métriques :

- **Durée moyenne** — Temps entre déclenchement et fin
- **Taux de succès** — % de pipelines vertes sur 7 jours
- **Tests flaky** — Tests qui échouent de façon aléatoire
- **Queue time** — Temps d'attente avant qu'un runner soit disponible

Alertes recommandées

- Pipeline qui dépasse X minutes (régression de performance)
- Taux de succès qui chute sous Y% (problème systémique)
- Job spécifique qui échoue plus de Z fois (test flaky à corriger)

Les invariants d'une pipeline

Au-delà des concepts techniques, une pipeline repose sur trois **garanties fondamentales**. Si l'une manque, vous avez un script qui s'exécute à distance — mais pas une vraie pipeline.

Reproductibilité : même entrée, même sortie

Règle d'or : Le même commit doit produire le même artefact, peu importe quand ou où vous l'exécutez.

Si votre pipeline donne des résultats différents à chaque exécution, **elle perd toute valeur**. On ne peut plus faire confiance à ce qu'elle produit.

Ce qui casse la reproductibilité :

- **Dépendances non épinglées** — `npm install` sans `package-lock.json` peut installer des versions différentes à chaque fois.
- **Tests qui dépendent de l'heure** — Un test qui vérifie « il est entre 9h et 18h » va échouer la nuit.
- **Appels à des APIs externes sans mock** — Si l'API change ou est indisponible, vos tests plantent aléatoirement.
- **Cache non déterministe** — Si le cache contient des fichiers d'une exécution précédente, le résultat peut varier.

Comment garantir la reproductibilité ?

- Épingler toutes les dépendances (`package-lock.json`, `Pipfile.lock`, `go.sum`)
- Utiliser des images Docker avec des tags fixes (pas `latest`)
- Isoler les tests (pas de dépendance à l'heure locale, à des données externes)

Traçabilité : remonter de l'artefact au commit

Quand un bug apparaît en production, vous devez pouvoir répondre à :

- **Quel commit** a introduit ce changement ?
- **Quelle pipeline** a construit cet artefact ?
- **Quels tests** ont été exécutés ?

- **Qui** a déclenché le déploiement ?

Sans traçabilité, le debugging devient de l'archéologie : vous fouillez dans les logs, les messages de commit, en espérant tomber sur le bon élément.

Ce qui garantit la traçabilité :

- Chaque artefact est **taggé avec le SHA du commit** qui l'a produit
- Chaque déploiement conserve un **lien vers la pipeline** d'origine
- Chaque job expose ses **logs complets** (pas juste « succès » ou « échec »)

Observabilité : savoir ce qui se passe

Une pipeline doit exposer son état de santé en temps réel. Pas seulement « ça marche » ou « ça plante », mais des métriques actionnables.

Pourquoi c'est critique ?

Parce qu'une pipeline qui ralentit progressivement finit par bloquer toute l'équipe. Si vous ne détectez pas qu'un job de test passe de 2 minutes à 15 minutes, vous le découvrirez trop tard — quand personne ne voudra plus pousser de code.

Ce que la pipeline révèle

Une pipeline CI/CD n'est jamais neutre. Elle **cristallise les décisions** de l'équipe, parfois de façon inconsciente.

Questions qu'une pipeline répond implicitement :

- **Quels tests sont obligatoires ?** — Si un test peut être ignoré, il n'est pas obligatoire. Si la pipeline bloque sans lui, il l'est.
- **Qui peut déployer en production ?** — Si tout le monde peut merger sur `main` et que ça déclenche un déploiement automatique, tout le monde peut déployer. Si ça nécessite une approbation, seuls certains rôles le peuvent.
- **Combien de temps peut-on attendre un feedback ?** — Une pipeline qui met 45 minutes à donner un retour décourage les petits commits fréquents. Une pipeline qui répond en 5 minutes encourage l'itération rapide.
- **Quelle confiance accorde-t-on aux dépendances externes ?** — Si la pipeline bloque dès qu'une dépendance ne répond pas, vous avez un point de défaillance unique. Si elle continue avec un cache ou une version épinglée, vous avez anticipé la panne.

La loi de Conway s'applique :

Une équipe cloisonnée (dev / ops séparés) produira une pipeline fragmentée (déploiement manuel après le build). Une équipe sans ownership clair produira une pipeline sans maintenance (personne ne nettoie les jobs obsolètes).